A Scalar Softmax Merge Unit With Selective Offloading for RISC-V Vector Clusters

Anonymous Author(s)

***Abstract*—Blockwise attention repeatedly merges a compact (m, l, O) state after each tile. Each online Softmax merge combines a state-dependent scalar recurrence with a regular $O[D]$ vector update. The Full-Offload ablation also assigns the vector update to dedicated hardware, which changes the specialization boundary. B2R is the matched-arithmetic RVV baseline: software executes the scalar recurrence while existing RVV executes the vector update. Proposed combines a Scalar Softmax Merge Unit (SMU) for the recurrence with existing RVV for the vector update. The SMU returns two weights through a tightly coupled data memory (TCDM)-mediated handoff; memory-mapped I/O (MMIO) starts the SMU and reports completion. Across three model-derived one-position merge shapes, Proposed reaches a 4.53× geometric-mean measured RTL/Verilator cycle-count ratio over B2R, reduces fitted C_s by 16.30×, while fitted C_v remains within 6.22% of B2R. The standalone Scalar SMU block uses 73,505 mapped cells and 77,103.3 Nangate45 Liberty cell-area units. The Full-Offload block requires 48.8% more standalone mapped Liberty area than the standalone Scalar SMU block. The timing flow supplies only PARTIAL pre-layout reg-to-reg combinational-delay proxies, so it cannot support a synchronous $F_{\max}$ claim. The evidence supports selective scalar offloading within the measured merge-kernel and standalone SMU block scope.**

***Index Terms*—online softmax, attention, RISC-V vector processor, scratchpad, accelerator**

I. Introduction

Tiled attention avoids materializing the full score matrix by updating online Softmax merge state after each tile [1], [2]. Each merge produces m_{new}, l_{new}, w_{old}, and w_{tile} before the regular $O[D]$ vector update. The recurrence contains row-level dependencies; each vector element update remains independent after the two weights become available. This split creates a scalar recurrence cost and a regular vector-update cost with different hardware requirements.

Existing designs occupy broad/full-offload and programmable/vector endpoints. Full designs assign normalization and vector work to dedicated datapaths, whereas programmable and RVV approaches retain processor-side execution [3], [4], [5], [6], [7]. This paper evaluates the intermediate boundary that specializes only the state-dependent recurrence while preserving the existing RVV update. The recurrence equations and EXP/reciprocal LUTs are implementation ingredients; the contribution is their dependency-aligned hardware placement.

Spatz provides RISC-V Vector (RVV) execution and shared tightly coupled data memory (TCDM) for the regular vector update [8]. B2R uses the matched-arithmetic software recurrence and existing RVV as the comparison baseline. The Full-Offload ablation assigns both phases to dedicated hardware and therefore duplicates the regular vector path.

Proposed assigns only the state-dependent recurrence to a Scalar SMU and keeps the existing RVV datapath for $O[D]$. The SMU writes the new scalar state and two weights to shared TCDM. Software waits for completion, loads the weights, and invokes existing RVV for the vector update (Figure 1). This boundary leaves the ISA and RVV datapath unchanged.

Across three model-derived one-position merge shapes, Proposed reaches a 4.53× geometric-mean measured cycle-count ratio over B2R. Proposed reduces fitted C_s by 16.30×, while fitted C_v remains within 6.22% of B2R. The Full-Offload ablation has 3.78× Proposed C_v and requires 48.8% more standalone mapped Liberty area. We make three contributions:

- **Scalar SMU.** Proposed assigns maximum selection, exponential rescaling, length accumulation, reciprocal normalization, and weight generation to a recurrence-only Scalar SMU with a nominal no-stall $13 + 1 + 1 + 9$ state schedule.
- **Unchanged-ISA integration.** Proposed retains the regular $O[D]$ update on existing RVV. TCDM provides the post-completion weight handoff without a direct SMU-to-RVV path.
- **Measured boundary result.** The fitted N-dependent coefficient falls by 16.30× while the fitted ND-dependent coefficient remains within 6.22% of B2R; Full-Offload remains an internal ablation.

II. Selective-Offload Architecture

Proposed is Scalar SMU plus existing RVV. The SMU executes the state-dependent online Softmax merge recurrence, while existing RVV executes the regular $O[D]$ update. Shared TCDM carries scalar inputs, new state, and weights between the software-visible phases. MMIO starts the SMU and reports completion; software then loads the weights and invokes RVV. Figure 1 shows this boundary and its TCDM-mediated handoff.

A. Specialization Boundary

$$m_{\text{new}} = \max(m_{\text{old}}, m_{\text{tile}}), \tag{1}$$

$$s_{\text{old}} = l_{\text{old}} \exp(m_{\text{old}} - m_{\text{new}}), \tag{2}$$

$$s_{\text{tile}} = l_{\text{tile}} \exp(m_{\text{tile}} - m_{\text{new}}), \tag{3}$$

$$l_{\text{new}} = s_{\text{old}} + s_{\text{tile}}, \tag{4}$$

$$w_{\text{old}} = s_{\text{old}}/l_{\text{new}}, \quad w_{\text{tile}} = s_{\text{tile}}/l_{\text{new}}, \tag{5}$$

$$O_{\text{new}}[j] = w_{\text{old}} O_{\text{old}}[j] + w_{\text{tile}} O_{\text{tile}}[j], \quad 0 \le j < D. \tag{6}$$

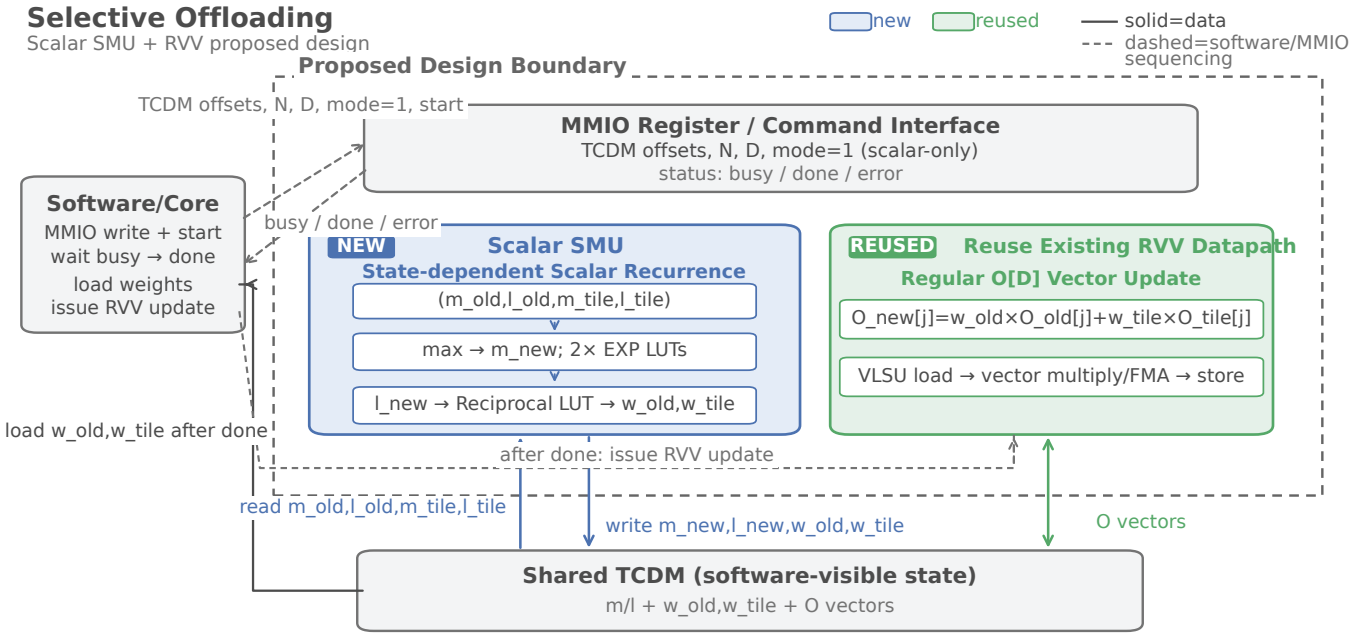

Fig. 1. Proposed architecture. Scalar SMU reads scalar state and writes m_{new}, l_{new}, w_{old}, and w_{tile} to shared TCDM. Software waits for completion, then invokes existing RVV for the $O[D]$ update. The TCDM-mediated handoff has no direct SMU-to-RVV link.

The first five scalar equations execute in Scalar SMU; existing RVV executes the final $O[D]$ update. They cover maximum selection, two exponential rescalings, length accumulation, and reciprocal weight generation. EXP and reciprocal LUTs approximate the arithmetic in RTL; the equations define the dataflow, not IEEE-exact arithmetic. Scalar SMU returns the new state and two weights, and RVV applies those weights independently for $0 \le j < D$.

The selected maximum controls both exponential arguments, and the new length controls reciprocal normalization. These dependencies serialize scalar work within each row. Each $O[j]$ update becomes independent after weight write-back. Scalar SMU therefore handles state-dependent decisions, while existing RVV retains regular element work and reuses its vector load/store unit (VLSU) and fused multiply-add (FMA) sequence. The selected maximum and new length stay outside the element loop.

B. Scalar Datapath and Schedule

The Scalar SMU datapath compares the two maxima and forms two deltas. Two EXP LUTs transform the deltas into scaled factors. Fixed-point multipliers combine each factor with l_{old} or l_{tile}, and an accumulator forms l_{new}. A reciprocal block generates w_{old} and w_{tile}. The scalar-only mode (mode 1) writes the scalar results to TCDM and exits before `UPDATE_VECTOR`; the two LUT results stay paired with their row lengths through normalization.

RTL observer counts define a nominal no-stall state schedule: `LOAD_SCALAR`=13, `COMPUTE_SCALAR`=1, `COMPUTE_WEIGHT`=1, and `STORE_SCALAR`=9. The nominal total is 24 SMU-busy cycles per row, with 22/24 cycles in TCDM communication states and 2/24 cycles in compute states. Model-shape observers record 289, 769, and 962 busy cycles for BERT, Mistral, and Qwen2.5-14B-Instruct, respectively, showing one or two TCDM wait cycles beyond the nominal schedule. These counts exclude software-visible command and completion overhead, the separate RVV phase, and the fitted C_s coefficient; they are not system latency.

C. TCDM-Mediated RVV Handoff

The scalar-only mode makes the handoff explicit in TCDM. For each row, Scalar SMU reads four FP32 inputs, m_{old}, l_{old}, m_{tile}, and l_{tile}, from TCDM, computes the new scalar state, and writes m_{new}, l_{new}, w_{old}, and w_{tile} back to TCDM. The weights are software-visible arrays, so there is no private weight state or weight bypass. Software programs the MMIO command, starts the scalar-only mode, waits for done, and then loads both weight arrays from TCDM. It invokes existing RVV to load $O_{\text{old}}[D]$ and $O_{\text{tile}}[D]$, applies the weights, and stores $O_{\text{new}}[D]$. There is no direct SMU-to-RVV path; completion and shared TCDM state mediate the transition, and RVV sees only completed rows. Rows remain aligned by their shared TCDM indexing.

MMIO provides command and status without changing instruction decoding. The cluster peripheral presents the selected operation to Scalar SMU, which uses an independent TCDM requester for scalar reads and writes. Existing core ports continue to serve RVV loads and stores, so the vector path remains unchanged and shared TCDM mediates arbitration without a private engine link. The ISA and RVV datapath remain unchanged. The evaluation separates measured cycles from standalone SMU evidence.

III. Evaluation

The evaluation measures the selective boundary at two levels. RTL/Verilator cycles quantify the merge-kernel path, and Yosys/ABC quantifies standalone SMU hardware. Proposed combines Scalar SMU with existing RVV; B2R is the matched-arithmetic RVV baseline; Full-Offload is an internal ablation. Speedup always means B2R measured cycles divided by design measured cycles.

A. Experimental Setup

We evaluate FP32 merge state on a Spatz cluster with (m, l, O) arrays resident in TCDM. The software reciprocal reference uses the same LUT scale as RTL. The matrix contains 26 deterministic cases, 3 configurations, and 3 reruns (234 records). Every record passes the matched-LUT scale-aware functional check at 10^{-3}, produces no nonfinite values, and reproduces the same result on rerun. These checks do not establish formal verification, IEEE-exact arithmetic, or model-level accuracy. Each cycle count measures one logical query-position merge-kernel work item in RTL/Verilator. For selected rows, N is query-head count and D is hidden width per query head; GQA uses query heads rather than KV heads. The rows use BERT $(12, 64)$, Mistral $(32, 128)$, and Qwen2.5-14B-Instruct $(40, 128)$ with deterministic generated states, not captured activations or end-to-end inference.

Standalone hardware evidence uses Yosys/ABC with the Nangate45 library. The area flow uses `abc -fast` to report mapped cells and Liberty cell-area units. A separate timing-driven flow reports a PARTIAL pre-layout Nangate45/ABC reg-to-reg combinational-delay proxy. The two mappings answer separate questions, so timing-flow cells and area do not enter the formal area result. The records contain no cluster PPA, power, energy, or synchronous $F_{\max}$ evidence.

B. Matched-Arithmetic RVV Baseline and Full-Offload Ablation

B2R supplies the matched-arithmetic RVV comparison because it and Proposed use the same regular RVV update. Software executes the scalar recurrence in B2R; Proposed moves only that recurrence to Scalar SMU. The Full-Offload ablation moves the vector update as well. The B2R/Proposed cycle ratio compares this specified implementation boundary and does not establish an optimized-software lower bound.

C. Scaling Mechanism

We fit $C = C_0 + C_sN + C_vND$ to 23 measured RTL/Verilator shapes for each design. Here C_0 is the fitted intercept, C_s is the fitted N-dependent cycles/row coefficient, and C_v is the fitted ND-dependent cycles/element coefficient. Fits have R^2 values from 0.99928 to 0.99999. Figure 2 plots these coefficients with C_s on a logarithmic axis and C_v on a linear axis. The fit describes cycle growth; the nominal FSM schedule measures a separate scalar-engine busy interval.

B2R fits $C_0/C_s/C_v = 161.353823/1471.818713/2.267063$, while Proposed fits $1382.770246/90.278155/2.126010$. The Full-Offload ablation fits $1285.577085/19.317262/8.029465$. Proposed reduces fitted C_s by 16.30×; its fitted C_v remains within 6.22% of B2R. The fitted coefficients describe cycle growth and do not represent FSM busy cycles or system latency.

The Full-Offload ablation lowers C_s further, but its fitted C_v reaches 3.78× Proposed. The component comparison places the larger ND-dependent coefficient on the implementation that assigns regular vector work to dedicated hardware, while Proposed retains existing RVV.

D. Model-Derived Shapes

We evaluate three deterministic generated one-position merge shapes: BERT $(N, D) = (12, 64)$, Mistral $(32, 128)$, and Qwen2.5-14B-Instruct $(40, 128)$. Table I reports measured RTL/Verilator merge-kernel cycles for B2R, Proposed, and the Full-Offload ablation. The table records one logical query position, not captured activations or end-to-end inference. Each role executes the same recurrence and vector-update shape.

The measured ratios are 4.81× for BERT, 4.38× for Mistral, and 4.41× for Qwen2.5-14B-Instruct, with a 4.53× geometric mean.

E. Hardware Cost and Full-Offload Ablation

Table II separates area and timing evidence. The area flow reports standalone mapped cells and unconstrained pre-layout Nangate45 Liberty cell-area units with `abc -fast`. The separate timing flow reports a PARTIAL pre-layout reg→reg proxy; it excludes clock-to-Q, setup/hold, skew, I/O constraints, and physical buffering/load semantics. The proxy cannot support synchronous $F_{\max}$ or rank the designs by timing.

The area flow maps the standalone Scalar SMU block to 73,505 cells and 77,103.3 Nangate45 Liberty cell-area units. The Full-Offload block maps to 107,372 cells and 114,713.3 units, 48.8% more than the Scalar SMU block.

Figure 3 compares merge-kernel cycle-count ratios with standalone accelerator-block area. The axes are separately scoped evidence, not cluster PPA or area efficiency. B2R has ratio 1.00× and zero incremental Scalar SMU block area; Proposed reaches 4.53× at 77.103 k units; Full-Offload reaches 1.89× at 114.713 k units. The ratio makes no frequency assumption.

Cluster PPA, power, and energy remain unavailable. These mappings bound the hardware claim to standalone block area; timing remains a PARTIAL combinational proxy. Full-Offload requires 48.8% more standalone mapped Liberty area and reaches a 1.89× geomean ratio versus Proposed's 4.53×.

IV. Related Work

Dedicated Softmax accelerators assign broad normalization work to specialized datapaths. Softermax combines unnormalized and normalization hardware, while TEA-S implements a compact PLAC-based full-Softmax architecture [3], [9]. SoftEx couples a full-Softmax/GELU HWPE to shared TCDM [4]. Fairoose and Mishra likewise realize parallel

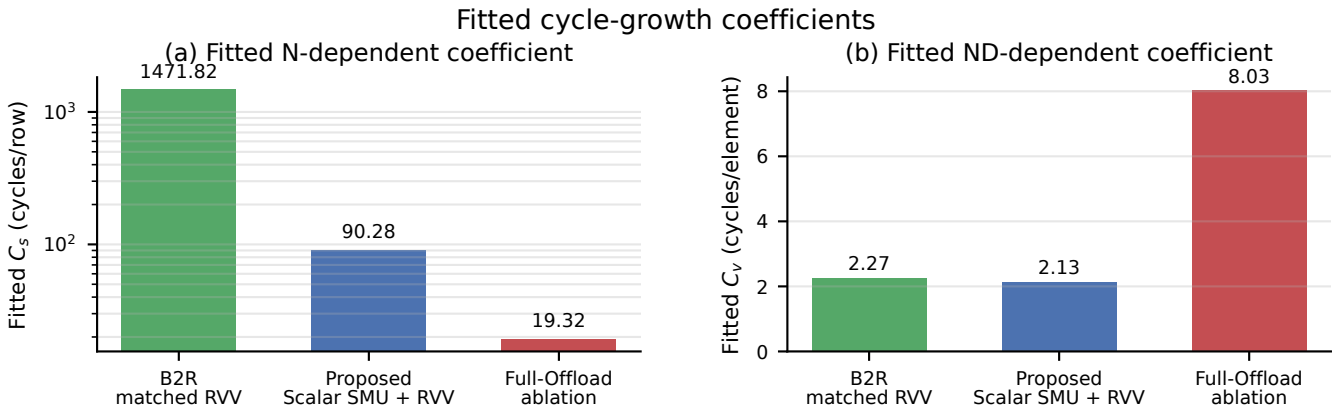

Fig. 2. Fitted N-dependent C_s and ND-dependent C_v coefficients from RTL/Verilator cycles. C_s uses a logarithmic axis; the nominal FSM schedule is a separate busy interval.

TABLE I
DETERMINISTIC GENERATED ONE-POSITION SHAPES AND MEASURED MERGE-KERNEL CYCLES. N IS QUERY-HEAD COUNT AND D IS PER-HEAD WIDTH; SPEEDUP IS B2R CYCLES DIVIDED BY PROPOSED CYCLES.

Workload	N	D	B2R	Proposed	Full-Offload	B2R/Proposed
BERT	12	64	19,856	4,128	7,704	4.81×
Mistral	32	128	56,296	12,866	34,728	4.38×
Qwen2.5-14B-Instruct	40	128	69,348	15,733	43,206	4.41×
Geomean	–	–	–	–	–	4.53×

TABLE II
STANDALONE SCALAR SMU AND FULL-OFFLOAD BLOCKS FROM SEPARATE FLOWS. AREA VALUES USE NANGATE45 LIBERTY CELL-AREA UNITS; DELAY VALUES ARE PARTIAL PRE-LAYOUT REG→REG PROXIES. FULL-OFFLOAD IS AN INTERNAL ABLATION.

Metric	Scalar SMU block	Full-Offload block
Mapped cells	73,505	107,372
Mapped Liberty area (Nangate45 Liberty cell-area units)	77,103.3	114,713.3
Relative Liberty area to Scalar SMU block	1.000×	1.488× (+48.8%)
PARTIAL reg→reg delay proxy	12.34 ns	13.20 ns

online normalization inside a full Softmax accelerator [10]. Even the cluster-local TCDM coupling in SoftEx does not isolate the state-dependent blockwise merge recurrence from the rest of Softmax. These designs therefore establish broad or full-Softmax hardware points, not the scalar-recurrence-only boundary evaluated here.

Full-attention hardware follows a second specialization boundary. Alexandridis et al. fuse exponential and vector multiplication operators in custom FlashAttention hardware [11]. This path keeps the output-vector work in dedicated hardware alongside recurrence-related computation. Its operator fusion reduces software work, but it does not reuse an existing RVV datapath through a scalar-only merge engine. This is a full-attention boundary with dedicated vector work. The boundary differs from a small recurrence engine because attention output production remains part of the hardware path.

Programmable and vectorized approaches retain the processor-side boundary. Spatz provides the RVV/TCDM substrate used here [8]. VEXP adds BF16 scalar/SIMD exponential ISA operations while the remaining Softmax stays in software [5]. Titopoulos et al. fully vectorize FlashAttention with standard RVV instructions and no custom instructions [6]. Lai et al. use standard RVV intrinsics for Softmax acceleration and explicitly provide no specialized accelerator [12]. VFA relieves FlashAttention vector operations through global-maximum pre-computation [7]. It changes the update schedule, whereas this work retains the established recurrence and specializes only its state-dependent hardware boundary. The evaluated boundary combines a recurrence-only Scalar SMU, a TCDM-mediated weight handoff, and reuse of the existing RVV update.

V. CONCLUSION

Selective scalar offloading assigns the state-dependent online Softmax merge recurrence to Scalar SMU and retains the

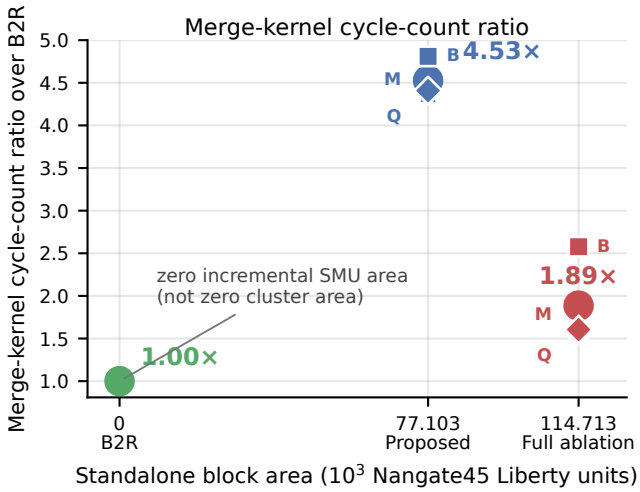

Fig. 3. Merge-kernel cycle-count ratio versus standalone accelerator-block area. The axes are separately scoped evidence, not cluster PPA or area efficiency. B2R at zero denotes zero incremental Scalar SMU block area. B, M, and Q identify BERT, Mistral, and Qwen2.5-14B-Instruct; circles are geomeans.

regular $O[D]$ update on existing RVV. Across three deterministic one-position shapes, Proposed reaches a 4.53× geometric-mean merge-kernel cycle-count ratio over matched-arithmetic B2R and reduces fitted C_s by 16.30× while fitted C_v remains within 6.22% of B2R. The standalone Scalar SMU block uses 73,505 mapped cells and 77,103.3 Nangate45 Liberty cell-area units. Full-Offload is an internal ablation with 48.8% more standalone mapped area and a 1.89× geomean ratio. Timing remains a PARTIAL pre-layout reg→reg proxy and cannot establish $F_{\max}$ or cluster performance. These results support selective scalar offloading within the measured merge-kernel and standalone block scope.

References

[1] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," *arXiv preprint arXiv:1805.02867*, 2018.

[2] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 16 344–16 359.

[3] J. R. Stevens, R. Venkatesan, S. Dai, B. Khailany, and A. Raghunathan, "Softermax: Hardware/software co-design of an efficient softmax for transformers," in *Proc. 58th ACM/IEEE Design Autom. Conf. (DAC)*, 2021, pp. 469–474.

[4] A. Belano, Y. Tortorella, A. Garofalo, L. Benini, D. Rossi, and F. Conti, "A flexible template for edge generative AI with high-accuracy accelerated softmax and GELU," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 15, no. 2, pp. 200–216, 2025.

[5] R. Wang, G. Islamoglu, A. Belano, V. Potocnik, F. Conti, A. Garofalo, and L. Benini, "VEXP: A low-cost RISC-V ISA extension for accelerated softmax computation in transformers," in *Proc. IEEE 32nd Symp. Comput. Arithmetic (ARITH)*, 2025, pp. 37–44.

[6] V. Titopoulos, K. Alexandridis, and G. Dimitrakopoulos, "Vectorized FlashAttention with low-cost exponential computation in RISC-V vector processors," *J. Supercomput.*, vol. 82, no. 4, 2026, art. no. 189.

[7] Y. Sun, Y. Li, Z. Zou, B. Du, Z. Zhang, H. Dong, G. Fan, and H. Wang, "VFA: Relieving vector operations in flash attention with global maximum pre-computation," arXiv:2604.12798, 2026.

[8] M. Perotti, S. Riedel, M. Cavalcante, and L. Benini, "Spatz: Clustering compact RISC-V-based vector units to maximize computing efficiency," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 7, pp. 2488–2502, 2025.

[9] Z. Mei, H. Dong, Y. Wang, and H. Pan, "TEA-S: A tiny and efficient architecture for PLAC-based softmax in transformers," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 9, pp. 3594–3598, 2023.

[10] S. Fairoose P. and A. Mishra, "A custom hardware accelerator for softmax function with parallel online normalization," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, pp. 1–13, 2026, early access.

[11] K. Alexandridis, V. Titopoulos, and G. Dimitrakopoulos, "Low-cost FlashAttention with fused exponential and multiplication hardware operators," in *Proc. IEEE Comput. Soc. Annu. Symp. VLSI (ISVLSI)*, 2025, pp. 1–6.

[12] C.-F. Lai, P.-C. Liu, S.-H. Huang, and H.-H. Chen, "RISC-V vectorized softmax acceleration for IoT edge inference systems," *IEEE Internet of Things Journal*, vol. 13, no. 12, pp. 25 890–25 901, 2026.